Technical Challenge Walkthrough

Multi-Stage Web Security, Cryptographic Analysis & Forensic Enumeration (Levels 0–8)

- **Author:** Archie Cwebile Magoro
- **Activity:** Unguided 33-Hour Industry Timed Benchmark Challenge
- **Facilitator:** Hosted on SensePost Academy / Orange Cyberdefense Challenge Infrastructure
- **Methodological Standards:** NIST SP 800-115 | OWASP Top 10 | CWE
- **Highest Milestone:** Level 8 (Active Cryptographic & Digital Forensics Analysis)



Made with GAMMA

1. Introduction and Methodology Overview

The objective of this assessment was to navigate a series of unguided web security, cryptography, and digital forensics challenges spanning Levels 0 through 8. Unlike guided classroom labs that provide step-by-step instructions, this environment was designed to evaluate real-world problem-solving, pattern recognition, and persistence when facing intentional dead ends (Weidman, 2014).

To approach each level systematically, I followed a four-step methodology:

1

Surface and Source Enumeration

Inspecting how the web page behaves visually, followed by examining the raw HTML source code, external JavaScript files, and network traffic using browser Developer Tools (DevTools) (OWASP, 2024).

2

Concept Identification

Identifying the underlying security mechanism being tested, such as Domain Name System (DNS) records, data encoding, symmetric encryption, or file metadata.

3

Exploitation and Scripting

Using command-line utilities (PowerShell and Windows CLI tools), CyberChef, or custom JavaScript snippets in the browser console to decode payloads and bypass client-side traps.

4

Verification and Documentation

Recording the exact commands, decoded tokens, and visual proof at every stage, while documenting intentional "red herrings" (decoys planted by the challenge creators to test enumeration discipline).

Stage	Core Security Concept Tested	Primary Tool or Technique Used	Status
Level 0	Public DNS Record Lookup (A-Record)	nslookup CLI utility	Completed
Level 1	Insecure Client-Side Password Check (CWE-602)	HTML Source Inspection (view-source)	Completed
Level 2	Base64 Data Encoding and URL Parameters	PowerShell curl and Base64 Decoding	Completed
Level 3	Multi-Part String and ASCII Reconstruction	JavaScript Console Evaluation	Completed
Level 4	Client-Side Popup Trap and DOM Bypass	Raw Source Code Extraction	Completed
Level 5	DNS Text (TXT) Record Harvesting	nslookup -type=txt CLI query	Completed
Level 6	AES Encryption and Key Brute-Forcing	Custom JavaScript Console Loop (CryptoJS)	Completed
Level 7	Visual Steganography and Decoy Filtering	DOM Media Audit and QR Code Extraction	Completed
Level 8	Image Forensics, ROT13 and File Analysis	ExifTool, CyberChef, Steghide, HxD and GIMP	In Progress (Documented)

Made with GAMMA

2. Level 0: DNS Record Enumeration

What Is Being Tested?

When humans browse the internet, we type readable domain names (such as google.com), whereas computers communicate using numerical IP addresses. The **Domain Name System (DNS)** acts as the directory of the internet by translating domain names into IP addresses (Mockapetris, 1987). Because these records are publicly accessible, an analyst can query a DNS server directly to discover the IP address assigned to a target subdomain.

My Approach and Execution

The Level 0 interface prompted for the IP address of a subdomain named backup.orangecyberdefense.academy. Rather than guessing paths on the web page, I opened my terminal and executed a direct query using nslookup, a standard command-line administration tool used to query DNS servers.

I executed the following command:

```
nslookup backup.orangecyberdefense.academy
```

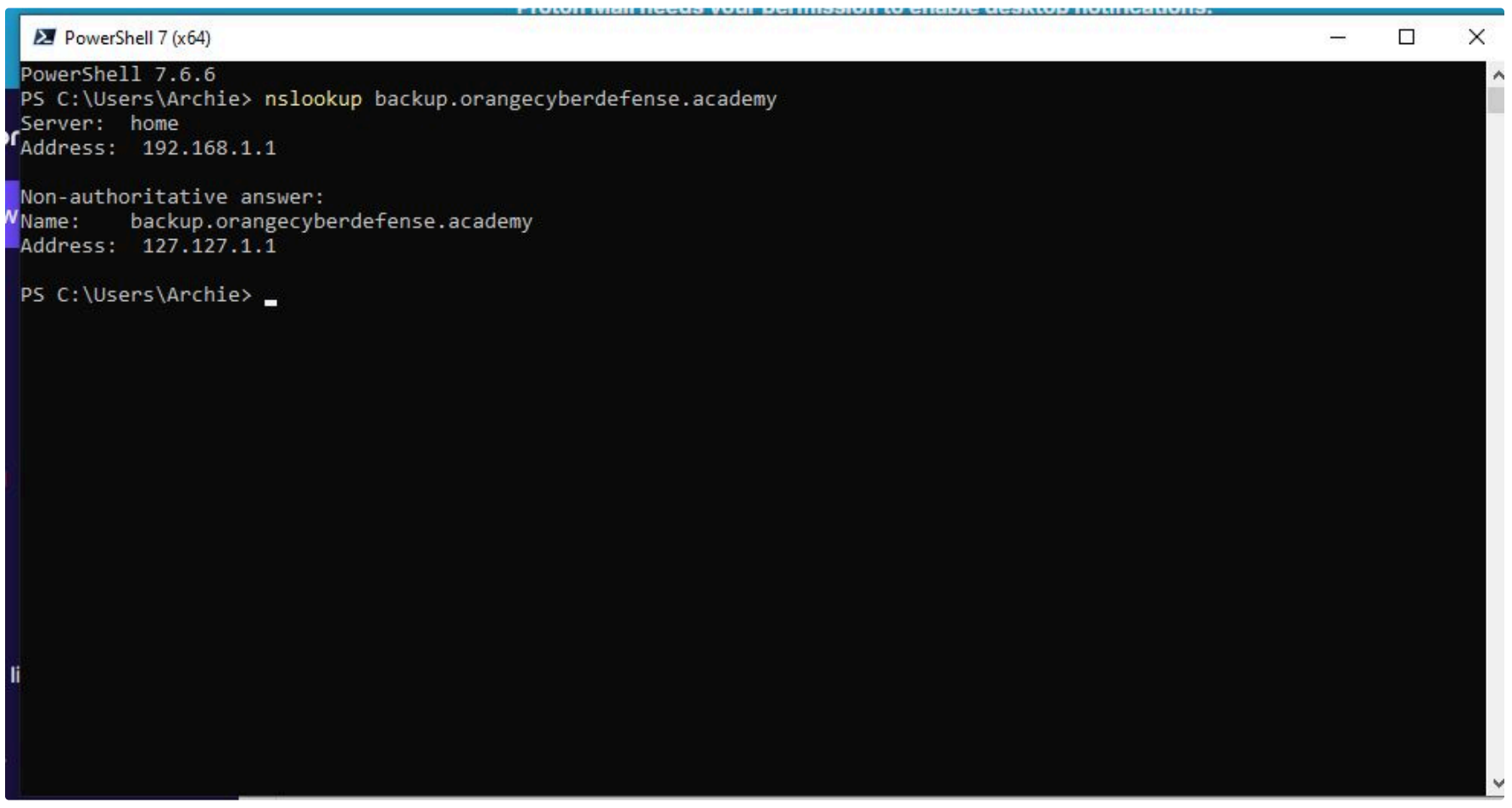
Breakdown of Command Parameters

Parameter	Meaning in Plain Language
nslookup	Instructs the operating system to query the configured DNS server for domain routing records.
backup.orangecyberdefense.academy	The specific target hostname we want to resolve into a numerical IPv4 address.

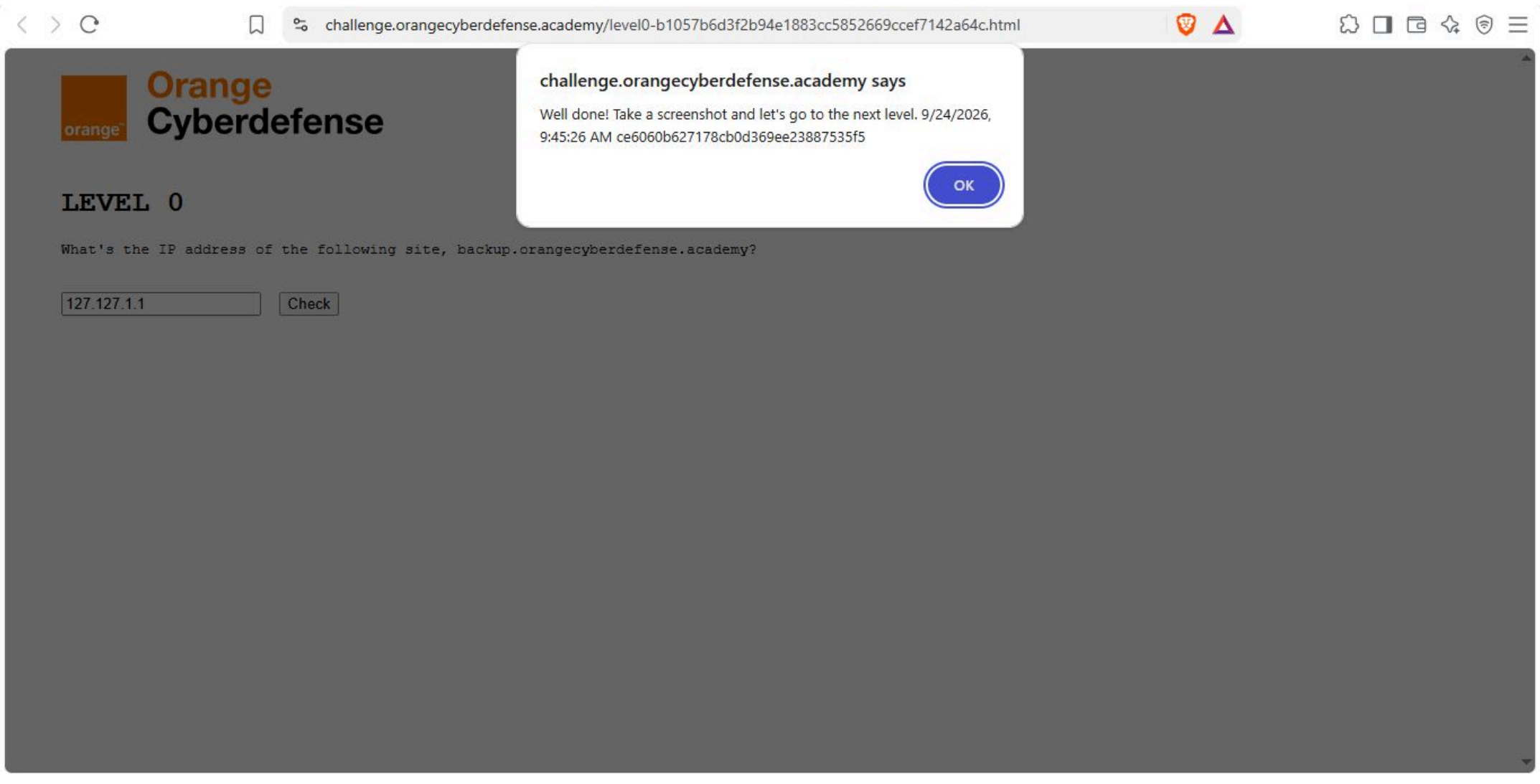
Results and Evidence

The DNS server responded with an **A-record** (Address Record) pointing to the loopback address 127.127.1.1. Submitting this IP address into the challenge input field unlocked the stage.

- **Screenshot 1 :** Terminal output showing nslookup resolving the target domain to 127.127.1.1.



- **Screenshot 2:** Confirmation modal displaying completion token ce6060b627178cb0d369ee23887535f5.



Made with GAMMA

3. Level 1: Client-Side Authentication Bypass

What Is Being Tested?

In a secure web application, when a user submits a password, that input is transmitted to a backend server where it is verified safely out of sight. A common architectural flaw is **Client-Side Enforcement of Server-Side Security (CWE-602)**, where the developer places the password check directly inside the browser's HTML or JavaScript code (MITRE Corporation, 2024). Because the user's browser must download this code to render the page, anyone can inspect the source file and read the password in plain text.

My Approach and Execution

When presented with the Level 1 login prompt, I bypassed the visual interface and inspected the raw HTML source code by navigating to: view-source:**https://challenge.orangecyberdefense.academy/level1-e137ac7cd5110d469a631a1cc2393227.html**

Inside the <script> tags, I located the JavaScript function responsible for validating the password:

```
function Try(passwd) {
  if (passwd == "Frame-Zoo-Fog-Package2-Choose") {
    alert("Well done!...");
    location.href = "level2_" + md5(passwd) + ".html";
  }
}
```

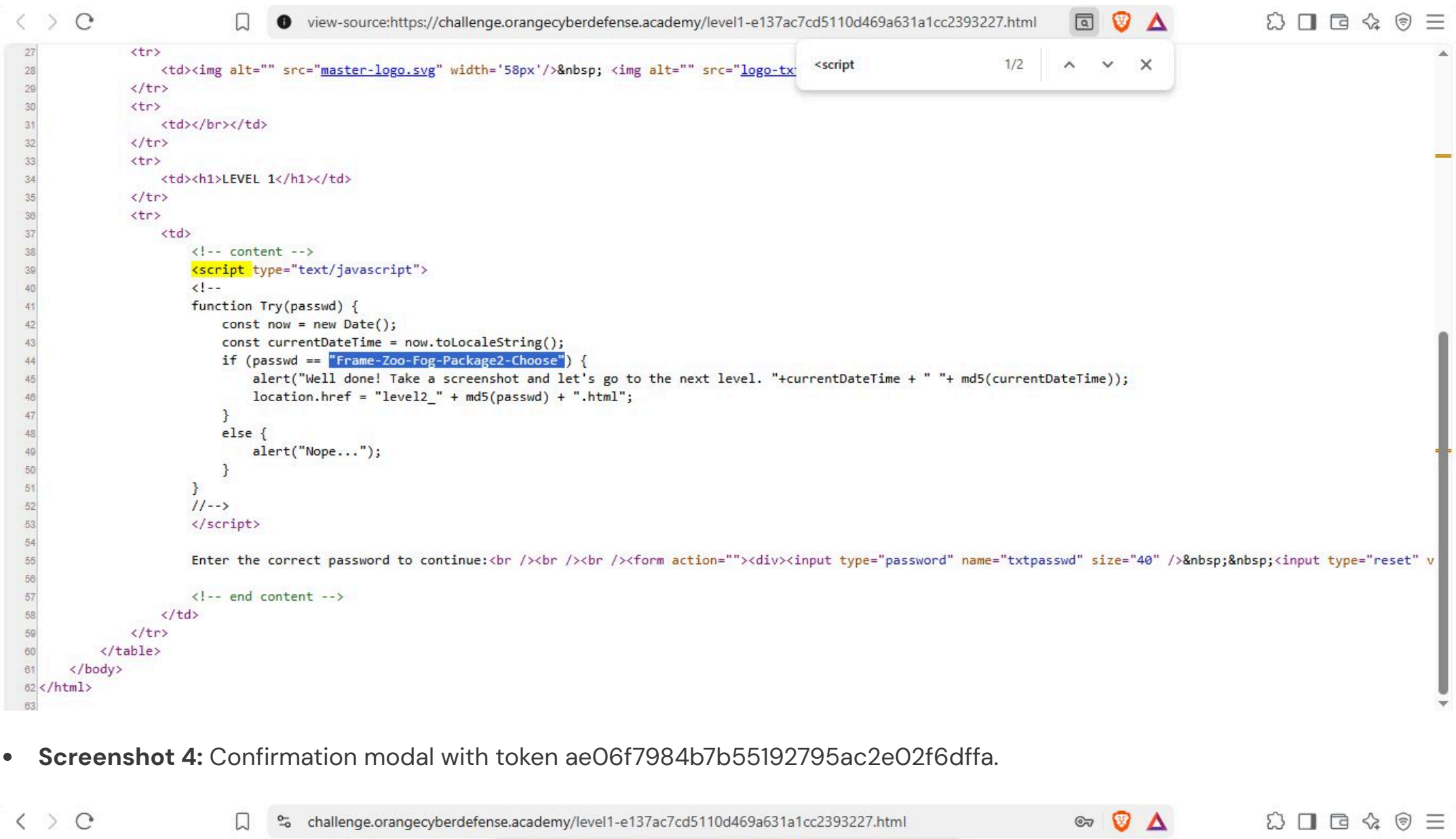
Plain-Language Code Breakdown

- **if (passwd == "Frame-Zoo-Fog-Package2-Choose"):** The browser checks whether the user's input matches the hardcoded string Frame-Zoo-Fog-Package2-Choose.
- **location.href = "level2_" + md5(passwd) + ".html":** Once matched, the script calculates the MD5 hash of the password (Rivest, 1992) and redirects the browser to the Level 2 page.

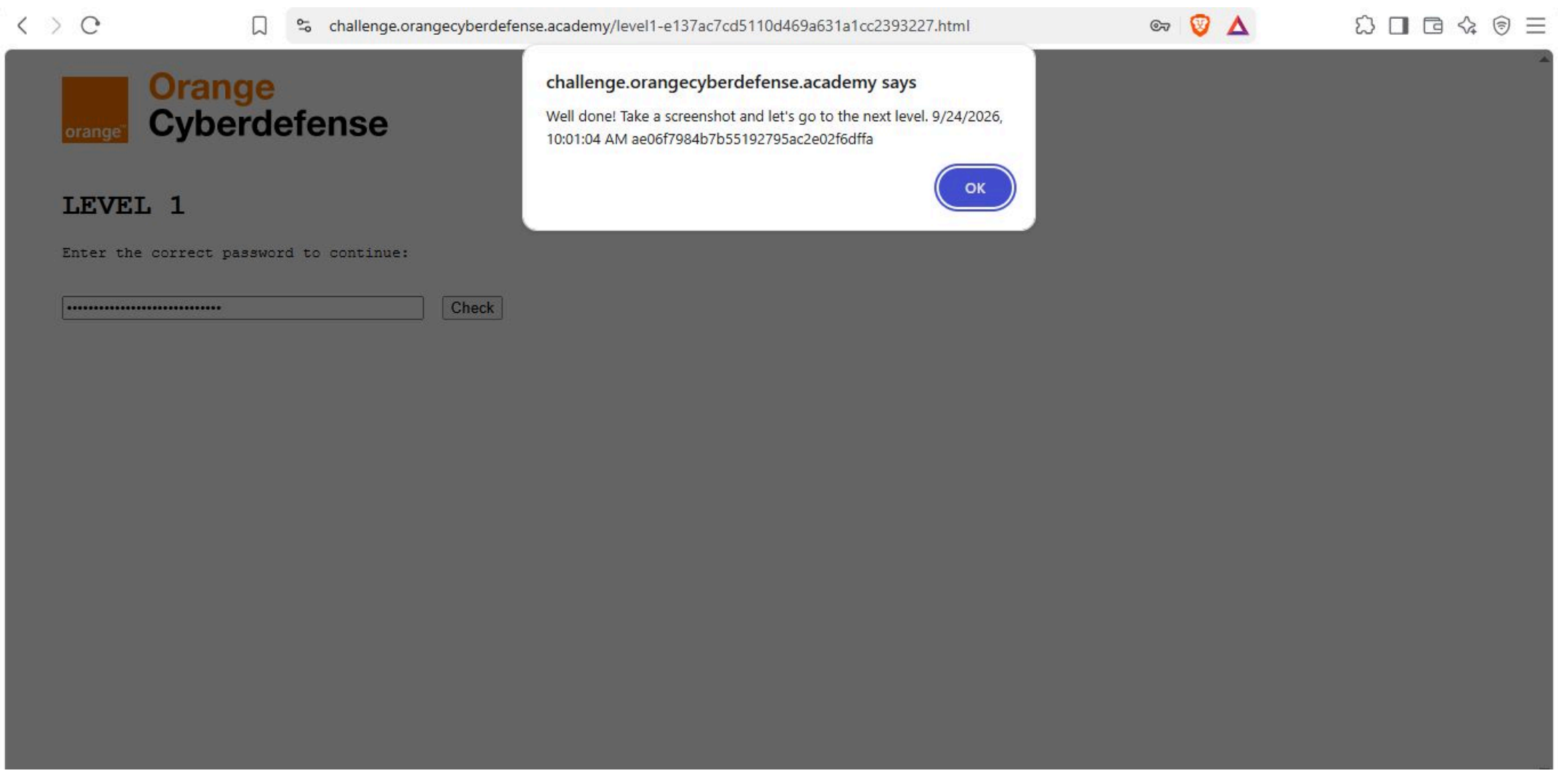
Results and Evidence

I copied the cleartext passphrase Frame-Zoo-Fog-Package2-Choose directly from the source code and submitted it into the form to advance.

- **Screenshot 3:** Browser source view revealing the hardcoded password inside the Try() function.



- **Screenshot 4:** Confirmation modal with token ae06f7984b7b55192795ac2e02f6dffa.



Made with GAMMA

4. Level 2 and Level 3: Data Encoding and String Reconstruction

What Is Being Tested?

Developers occasionally attempt to hide sensitive strings by **encoding** them (converting text into formats such as Base64 or ASCII character numbers) rather than encrypting them (Josefsson, 2006). Unlike encryption, which requires a secret mathematical key to unlock, encoding simply translates data into a standard format for safe transmission across systems. Anyone who recognises the encoding scheme can reverse it back into readable text immediately.

Level 2 Approach and Execution

On level2_3cc9c42d4a009ecb23fb8c5500ae5b35.html, I retrieved the page source and analysed its linked external JavaScript file (myapp.js). I identified a Base64-encoded validation routine. Decoding the Base64 string revealed the plain-text passphrase:

`base_64_is_encoding`

When submitted, the application appended the verified password to the URL bar as a GET query parameter: level2_3cc9c42d4a009ecb23fb8c5500ae5b35.html?txtpasswd=base_64_is_encoding

Level 3 Approach and Execution

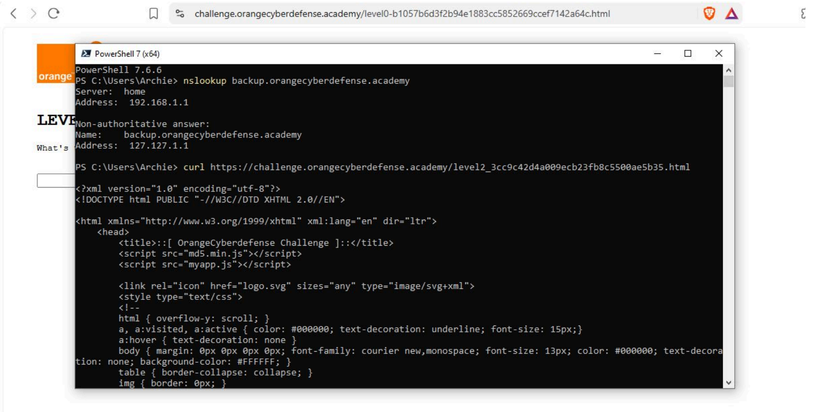
Level 3 (level3-base_64_is_encoding.html) expanded on this concept. Instead of storing a single encoded word in the code, the script split the password across multiple variables using a combination of Base64 strings, ASCII character codes, and substring slicing.

By evaluating those string operations together in my browser's Developer Console, I reconstructed the complete hidden phrase:

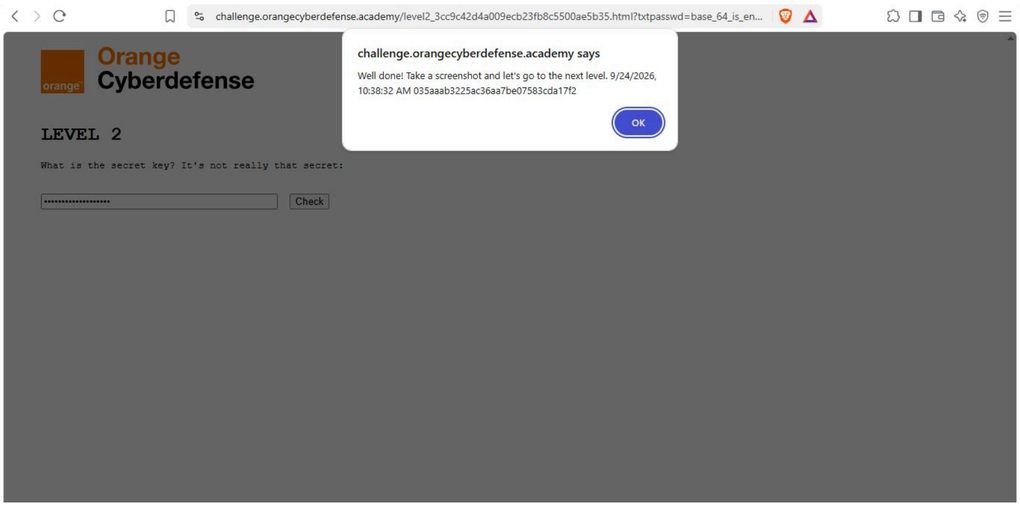
`The_Obstacle_isthe_way`

Submitting The_Obstacle_isthe_way authenticated the stage and routed the browser to Level 4.

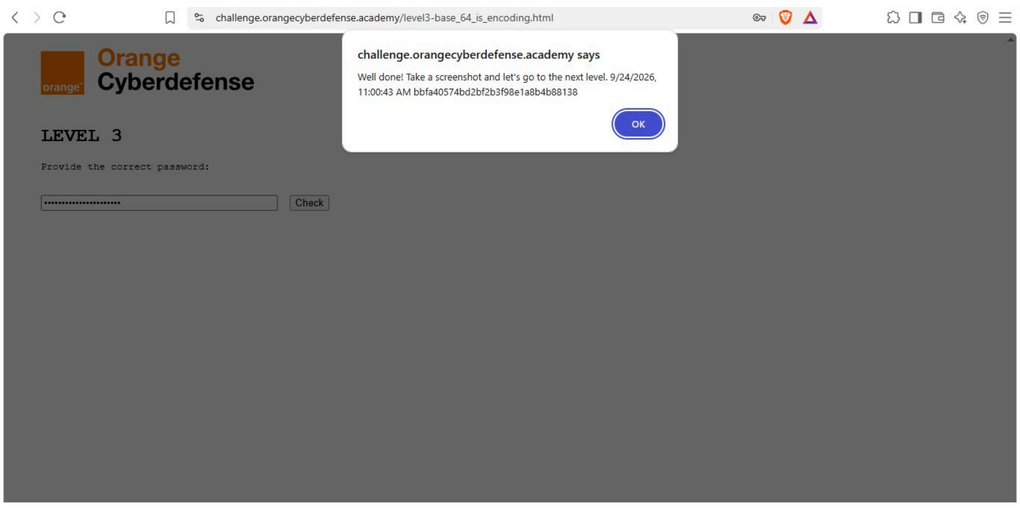
- **Screenshot 5:** Retrieving the raw source code via the PowerShell command line.



- **Screenshot 6:** Confirmation alert displaying token 035aaab3225ac36aa7be07583cda17f2.



- **Screenshot 7:** Confirmation modal displaying token bbfa40574bd2bf2b3f98e1a8b4b88138.



Made with GAMMA

5. Level 4 and Level 5: DOM Trap Bypass and DNS TXT Harvesting

Level 4: Bypassing a Client-Side Browser Trap

What Is Being Tested? Puzzle websites and malicious pages frequently use JavaScript execution loops (such as endless popups or redirecting buttons) to lock a user's browser window so they cannot interact with the page normally (OWASP, 2024). However, because these traps only execute *after* the browser renders the visual interface, an analyst can bypass them by inspecting the raw HTML source code directly.

My Execution: On level4-The_Obstacle_isthe_way.html, rather than interacting with the trapped button on screen, I inspected the raw HTML source code and located the destination URL hardcoded inside the Try() function:

```
function Try() {
  location.href = "level5-56ff181fdf03832f9a001d1b98568c98.html";
}
```

I copied level5-56ff181fdf03832f9a001d1b98568c98.html and pasted it directly into my browser address bar, bypassing the execution loop entirely.

Level 5: Harvesting Hidden DNS TXT Records

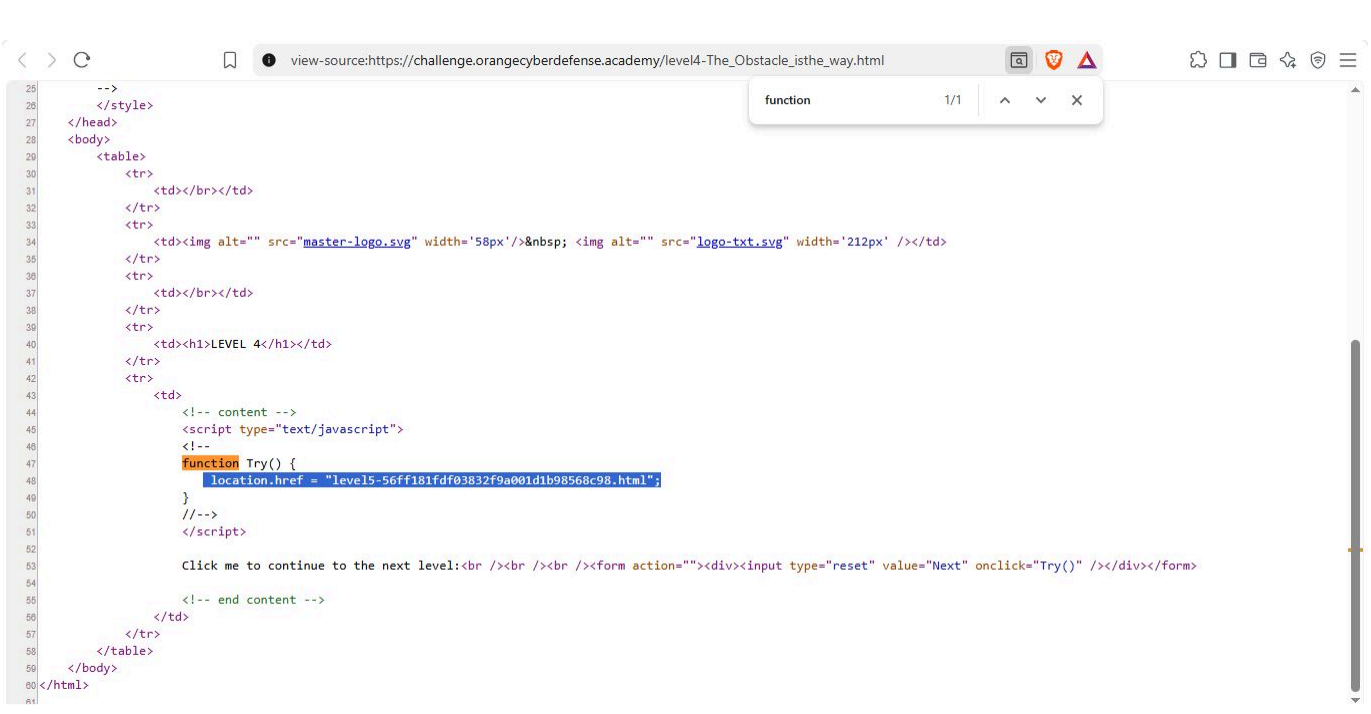
What Is Being Tested? In Level 0, we queried a standard A record to find an IP address. DNS servers also support **TXT (Text) Records**, which system administrators use to store human-readable notes, domain ownership verification codes, or email security policies (Mockapetris, 1987). A standard nslookup query does not display TXT records by default; the analyst must explicitly request the TXT record type.

My Execution: I opened PowerShell and executed nslookup with the -type=txt parameter to retrieve all text records attached to the challenge domain:

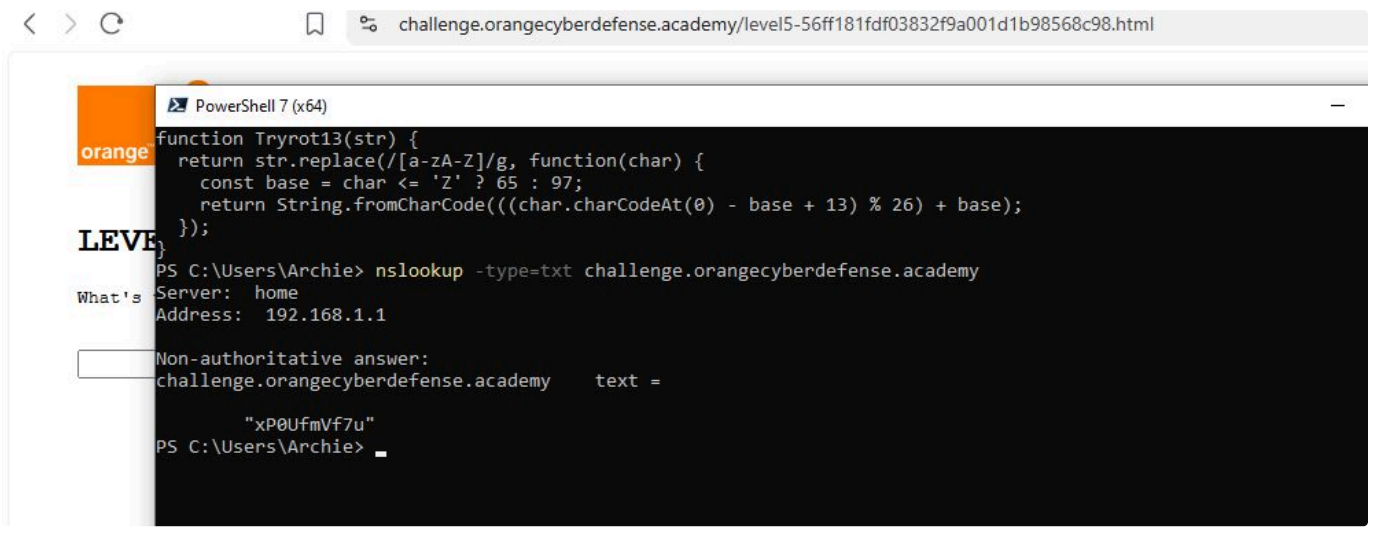
```
nslookup -type=txt challenge.orangecyberdefense.academy
```

The DNS server returned the hidden text string: "xPOUfmVf7u". Entering xPOUfmVf7u into the challenge prompt unlocked Level 6.

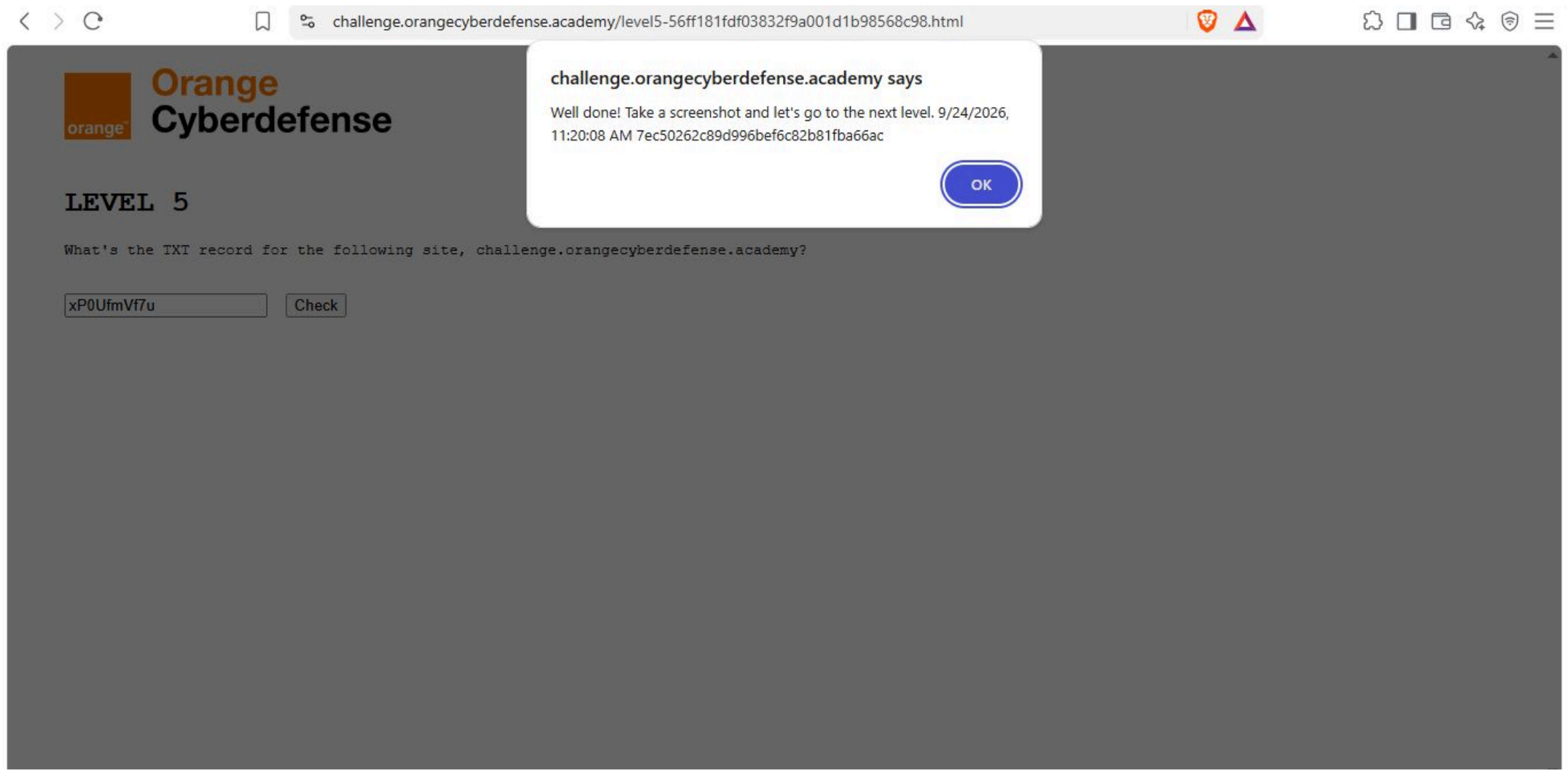
- **Screenshot 8:** Source code revealing the hardcoded Level 5 filename.



- **Screenshot 9:** Terminal output showing the extracted TXT record "xPOUfmVf7u".



- **Screenshot 10:** Confirmation modal displaying token 7ec50262c89d996bef6c82b81fba66ac.



Made with GAMMA

6. Level 6: AES Cryptanalysis and Automated Key Brute-Forcing

What Is Being Tested?

Unlike Base64 encoding (which anyone can reverse without a key), **AES (Advanced Encryption Standard)** is a symmetric cryptographic algorithm that requires a secret key to decrypt (NIST, 2001). An analyst cannot read an AES-encrypted payload (U2FsdGVkX1+S2mYCllFYKf...) without the key. However, if the secret key is derived from a small, predictable keyspace (such as a 5-digit number), a script can test candidate keys rapidly until the message unlocks (Erickson, 2008).

My Approach and Execution

Level 6 presented an AES-encrypted string alongside a 9x9 Sudoku grid containing five highlighted red squares representing the 5-digit decryption key. While solving the Sudoku constraints isolates the digits, I also observed that the page source loaded the CryptoJS library and verified whether the decrypted plaintext matched a target MD5 hash (3528b6fc9dd348938c9acdec61d06834).

To automate verification, I executed a JavaScript routine in the browser DevTools Console to test the derived Sudoku key (34278) directly against the AES ciphertext:

```
let decr = CryptoJS.AES.decrypt(enc, "34278").toString(CryptoJS.enc.Utf8);
if (md5(decr) === "3528b6fc9dd348938c9acdec61d06834") {
  console.log("DECRYPTED TEXT: " + decr);
}
```

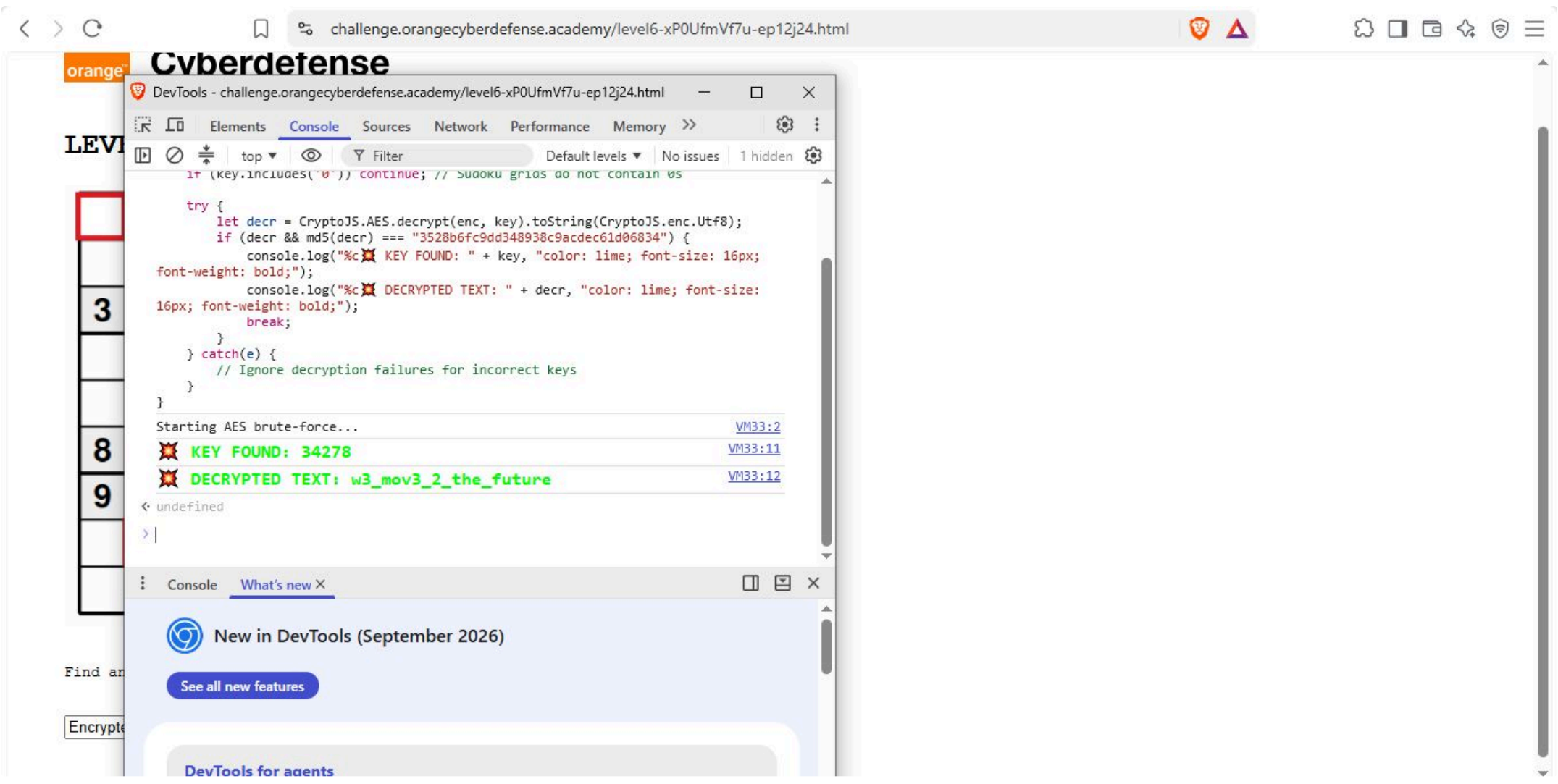
Plain-Language Code Breakdown

- **CryptoJS.AES.decrypt(enc, "34278"):** Attempts to unlock the encrypted payload (enc) using the 5-digit key 34278.
- **.toString(CryptoJS.enc.Utf8):** Converts the decrypted byte stream back into readable UTF-8 English text.
- **if (md5(decr) === "3528b6fc9dd348938c9acdec61d06834"):** Confirms that the MD5 fingerprint of the unlocked text matches the expected fingerprint of the solution.

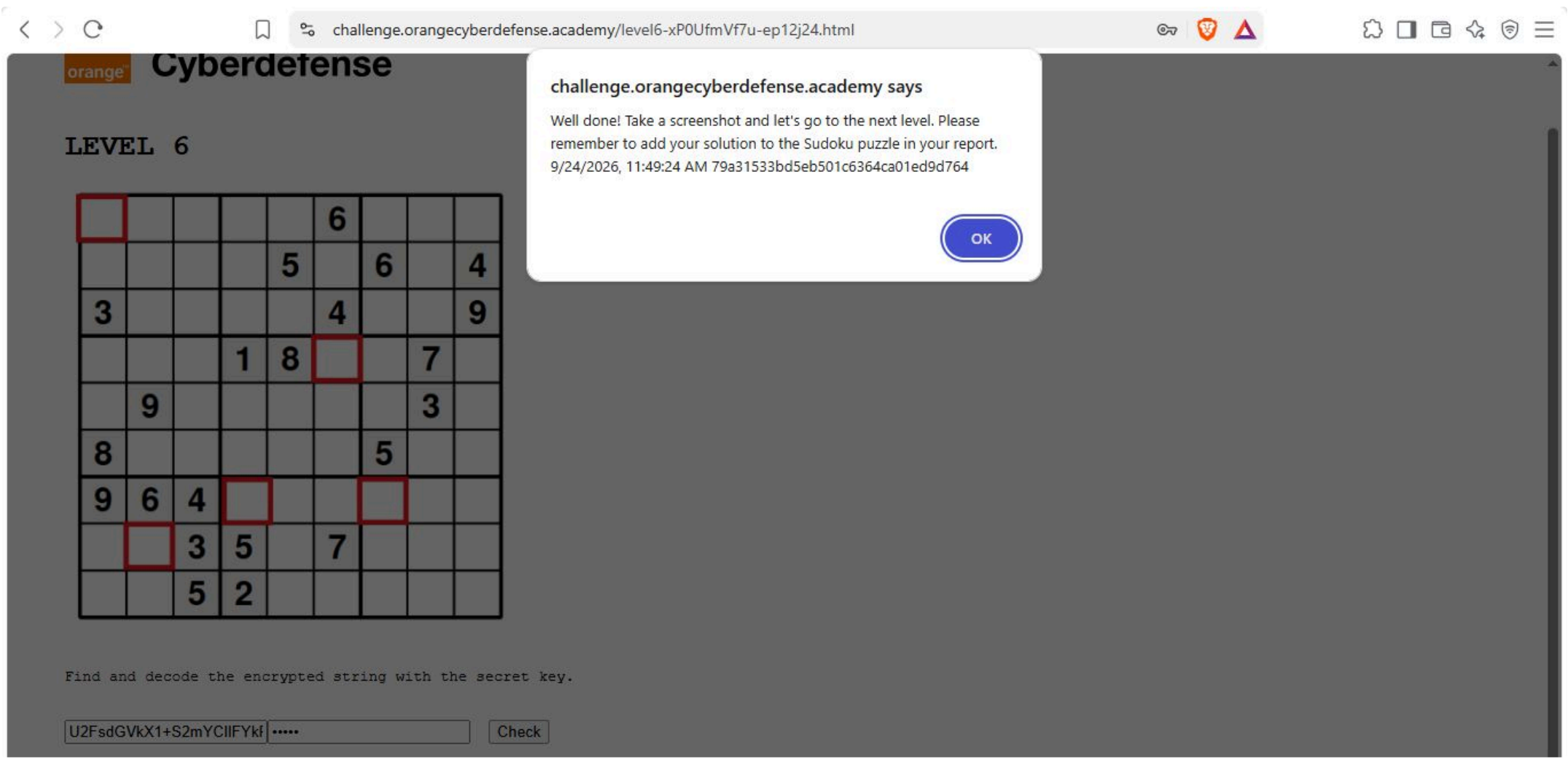
Results and Evidence

The console confirmed that 34278 was the valid key and output the decrypted secret phrase: **w3_mov3_2_the_future**.

- **Screenshot 11:** DevTools console script isolating key 34278 and printing the decrypted plaintext.



- **Screenshot 12:** Confirmation modal displaying token 79a31533bd5eb501c6364ca01ed9d764.



Made with GAMMA

7. Level 7: Filtering Red Herrings and Visual QR Steganography

What Is Being Tested?

During security assessments and capture-the-flag exercises, analysts frequently encounter **Red Herrings**: decoy data intentionally planted to resemble a vulnerability or cipher so the assessor wastes time pursuing dead ends. **Steganography** is the practice of concealing secret information inside an ordinary-looking image file or visual layout (Provos and Honeyman, 2003).

My Approach and Execution

Level 7 displayed a large HTML table populated with famous quotes, ROT13-shifted strings, AES ciphertexts, and JavaScript timestamp functions. On the surface, it appeared to be another complex cryptography stage.

Rather than attempting to crack every decoy string in the table, I audited the underlying HTML tags and media files embedded within the table cells. Buried inside one of the cells, I discovered an obscure image file named **23jkhqe30.png**.

When I isolated and opened 23jkhqe30.png directly, I observed that it was a **low-contrast QR code** (white squares rendered against a pale grey background so it remained nearly invisible against the page). Scanning the extracted QR code bypassed every decoy cipher on the page and revealed the direct URL for Level 8:

```
level8-b65079c27a33adc0b65d7038af7071ab.html
```

- **Screenshot 13:** Isolating the hidden low-contrast QR code (23jkhqe30.png) from the HTML table structure.



Made with GAMMA

8. Level 8: Reconnaissance, Metadata and Cryptographic Analysis

8.1 Initial Reconnaissance and Source Code Review

Level 8 (level8-b65079c27a33adc0b65d7038af7071ab.html) presented a static page containing a single image (laptop244244.jpg) depicting a hooded figure at a laptop displaying a simulated "Log In" screen, alongside a <script src="md5.min.js"></script> include and no interactive elements.

I reviewed the HTML source directly via browser **View Source** and confirmed the page consists only of a logo header, a level title, and the single image. There are no visible forms, input fields, or additional scripts beyond a benign setInterval timer at the bottom of the body:

```
const intervalId = setInterval(() => {
  const randomNumber = Math.random();
  console.log(randomNumber);
}, 5000);
```

Plain-Language Assessment: This script logs a random decimal number to the browser console every 5 seconds (5000 ms) and was immediately assessed as a non-functional distractor.

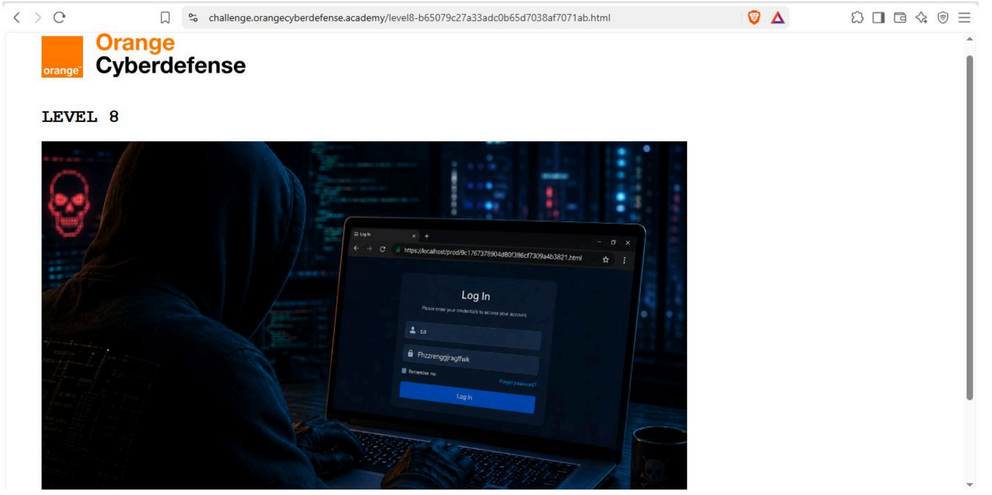
8.2 Metadata Analysis (ExifTool)

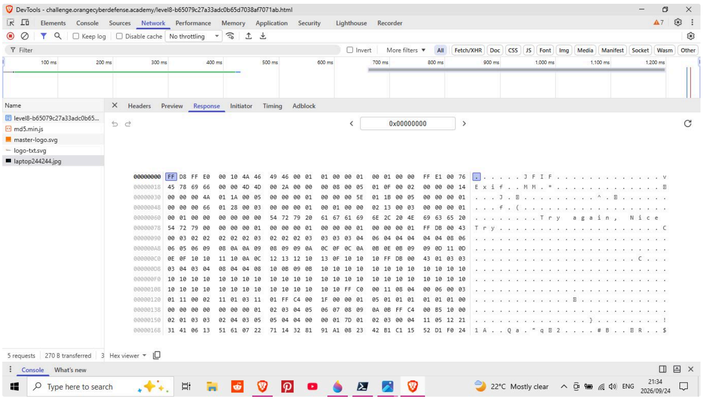
In digital forensics, **EXIF Metadata** refers to hidden header tags embedded inside image files that store camera manufacturer details, timestamps, and comments. I ran the Windows build of exiftool (and inspected the raw DevTools network response) against laptop244244.jpg.

The scan returned a single non-standard EXIF field:

> **Make: Try again, Nice Try**

- **Screenshot 14:** Metadata and network response inspection revealing the Make: Try again, Nice Try decoy tag.





8.3 Visual and Cryptographic Analysis of the Mock Login Screen

Because no login form exists on the HTML page itself (the "Log In" screen is part of the static JPEG image only), I ruled out direct credential submission on the current page and zoomed into the rendered mockup inside laptop244244.jpg to extract the displayed credentials table:

Visual Element	Extracted String	Technical Analysis and Verification
Username	sa	Assessed as a reference to the default Microsoft SQL Server **System Administrator** account, a well-known default credential in enterprise security contexts.
Password Ciphertext	Fhzzrenggjraglfwk	Assessed as ciphertext. I brute-forced all 26 possible Caesar cipher shifts against the string using **CyberChef's Caesar cipher recipe**. Shift 13 (**ROT13**) produced the only coherent output: **Summerattwentysjx**, beginning with the recognisable word "Summer". I verified the decode independently via manual re-checking of the ciphertext using zoomed screenshots to rule out transcription error.
Mockup URL Hash	9c1767378904d80f386cf7309a4b3821.html	Tested against CrackStation rainbow tables, confirming the 32-character hex string does not resolve to a standard unsalted word.

Made with GAMMA

9. Level 8: URL Hypothesis Testing, Steganography and File Checks

9.1 URL Pattern Hypothesis Testing

Noting that the live URL follows a level{N}-{token}.html structure, I investigated whether the token for Level 9 (level9-...) could be derived from the Level 8 credentials:

- **Direct Plaintext Tokens:** Tested the decoded password (plain and lowercase) directly as the next-level token, which returned HTTP 404 responses.
- **MD5 Hash Permutations:** Computed MD5 hashes of the decoded password, the raw ciphertext (Fhzzrenggjraglfwk), and username/password combinations using the page's own console md5() function (md5.min.js), testing each as the next-level token (all returned 404):

```
const candidates = [
  "sa", "Summerattwentysjx", "summerattwentysjx",
  "Summerattwentysix", "Fhzzrenggjraglfwk",
  "sa:Summerattwentysjx", "244244"
];
candidates.forEach(c => console.log(c, "=> level9-" + md5(c) + ".html"));
```

Cross-Level Validation: Cross-validated against the known Level 6 to Level 7 transition, which used a plaintext leetspeak phrase (w3_mov3_2_the_future) rather than a hash, and the Level 7 to Level 8 token, which was not an MD5 hash of the Level 7 answer. This confirmed that the URL token format is not standardised across levels and cannot be reverse-engineered by hashing alone.

9.2 Steganography Investigation (steghide)

Given the JPEG file format, I tested for Discrete Cosine Transform (DCT) steganography (Provos and Honeyman, 2003) using the **Windows build of steghide**, manually testing a curated set of candidate passphrases (decoded password variants, the username sa, 244244, and thematic terms), with no data extracted.

Methodological Note: A larger-scale automated wordlist attack (steghide / stegseek with a combined dictionary) was also explored during this investigation inside a separate Linux-based environment under time constraints, and has not yet been independently reproduced locally. This check is noted for completeness but excluded from the confirmed findings above pending re-verification.

9.3 File-Structure, Tonal Range (GIMP) and Linked Asset Review

- **Hex Editor Inspection (HxD):** Opened the raw laptop244244.jpg file in the **HxD** hex editor to check for trailing data appended after the JPEG End-of-Image (FF D9) marker or embedded file signatures and none were found.
- **Shadow and Tonal Range Compression (GIMP):** I used **GIMP's Levels tool** to compress the shadow and dark tonal range across the full image, checking for a low-contrast hidden watermark or QR code. This revealed a decorative circuit-pattern texture on the subject's hoodie, which was assessed as a stock-photo design element rather than a hidden message.
- **Linked SVG Asset Review:** I inspected all additional assets loaded by the page (master-logo.svg and logo-txt.svg) via the DevTools **Network** tab. Both contained only standard vector path data with routine Adobe Illustrator export metadata and no hidden comments.

Made with GAMMA

10. Conclusion, Current Status and References

Conclusion and Current Status

Across Levels 0 through 8, this assessment required applying structured enumeration across multiple cybersecurity domains:

- **Network and Infrastructure Reconnaissance:** Querying DNS servers with `nslookup` to extract both standard `A` records and hidden `TXT` records.
- **Web Application Reverse Engineering:** Auditing raw HTML and external JavaScript files via View Source and DevTools to bypass client-side password checks and browser execution traps.
- **Cryptography and Scripting Automation:** Reversing Base64 and ASCII encodings, brute-forcing Caesar cipher shifts in CyberChef to decode ROT13 strings, and writing custom JavaScript console routines to unlock AES-encrypted payloads.
- **Digital Forensics and Methodology:** Extracting low-contrast QR codes from DOM tables, identifying planted EXIF red herrings (`Make: Try again, Nice Try`) via Windows ExifTool, testing candidate passphrases in Steghide, inspecting raw End-of-Image (`FF D9`) byte boundaries in HxD, and compressing shadow tonal ranges in GIMP. While the displayed Level 8 credentials (`sa` and `Summerattwentysjx`) have been cryptographically decoded with high confidence, the mechanism for progressing to Level 9 remains under active investigation.

References

- Erickson, J. (2008) *Hacking: The Art of Exploitation*. 2nd edn. San Francisco: No Starch Press.
- Josefsson, S. (2006) *RFC 4648: The Base16, Base32, and Base64 Data Encodings*. Internet Engineering Task Force (IETF). Available at: **https://datatracker.ietf.org/doc/html/rfc4648** (Accessed: 25 September 2026).
- MITRE Corporation (2024) *CWE-602: Client-Side Enforcement of Server-Side Security*. Available at: **https://cwe.mitre.org/data/definitions/602.html** (Accessed: 25 September 2026).
- Mockapetris, P. (1987) *RFC 1035: Domain Names - Implementation and Specification*. Internet Engineering Task Force (IETF). Available at: **https://datatracker.ietf.org/doc/html/rfc1035** (Accessed: 25 September 2026).
- National Institute of Standards and Technology (NIST) (2001) *FIPS PUB 197: Advanced Encryption Standard (AES)*. Gaithersburg: NIST.
- Open Worldwide Application Security Project (OWASP) (2024) *OWASP Web Security Testing Guide (WSTG)*. Available at: **https://owasp.org/www-project-web-security-testing-guide/** (Accessed: 25 September 2026).
- Provos, N. and Honeyman, P. (2003) 'Hide and seek: An introduction to steganography', *IEEE Security & Privacy*, 1(3), pp. 32-44.
- Rivest, R. (1992) *RFC 1321: The MD5 Message-Digest Algorithm*. Internet Engineering Task Force (IETF). Available at: **https://datatracker.ietf.org/doc/html/rfc1321** (Accessed: 25 September 2026).
- Weidman, G. (2014) *Penetration Testing: A Hands-On Introduction to Hacking*. San Francisco: No Starch Press.

Made with GAMMA